

NewzWale - What It Is, and How It Was Built

A complete build document, written the way [compilefuture.com's "how to earn using AI" guide](#) lays out its micro-tool projects: what the site does, why it's built the way it is, and the actual steps that got it from an empty folder to a live Cloudflare Worker.

1. What NewzWale Is

NewzWale ("Only Facts News") is a live Indian news aggregator paired with an AI-grounded fact-checking tool. It is not a news publisher - it doesn't write articles. It does two things:

1. **Pulls real headlines** from Indian news sources in real time, organized by category (Top, India, World, Business, Sports, Entertainment, Technology, Health) and language (13 Indian languages), and links out to the original publisher for the full story.
2. **Checks whether a claim is true**, using a three-stage pipeline that looks for a published fact-check first, then live web evidence, and only ever answers when it actually found something to base an answer on.

The name and tagline ("Only Facts News") describe the whole product thesis: don't invent, don't guess, don't paraphrase news bodies you don't have rights to - surface real headlines and verify real claims, nothing more.

Core features

Feature	What it actually does
Live headline feed	Fetches per category/language from NewsData.io, cached 20 minutes in Cloudflare KV, falls back to an RSS reader if the API is unavailable
Fact Check Explorer (/verify)	Paste a headline, claim, or article URL -> returns Verified, Misleading, False, or Not enough evidence to judge - never a guess
Category browsing	/category/<slug> pages, one per news category
Language switch	Changes which language the <i>headlines</i> are fetched in (not the site UI - that stays English by design, and says so)
Save / bookmark	Per-article Save toggle, backed by <code>localStorage</code> , with a slide-in "Saved articles" drawer

Customize topics	Reader can show/hide which category rails appear on the homepage, also <code>localStorage</code> -backed
Most Read sidebar	Sticky sidebar surfacing trending headlines alongside quick topic-toggle chips
Live market ticker	Sensex/Nifty in the masthead - hides itself entirely if the upstream quote fails, rather than showing a stale or fake number
Dark mode	Full token-based theme, not a handful of overridden colors
Search	Client-side filter over already-loaded headlines from the hero search bar (no fake "search the internet" promise - it searches what's on the page)

2. Tech Stack

Layer	Choice	Why
Framework	Astro (SSR, <code>output: 'server'</code>)	Ships almost no JS by default; islands only where interactivity is needed (theme toggle, tabs, fact-check form)
Hosting	Cloudflare Workers via <code>@astrojs/cloudflare</code>	Free tier, edge-deployed, and the KV/AI bindings below live in the same platform
Styling	Tailwind CSS v4 with a custom <code>@theme</code> token system	One design-token source of truth; dark mode is a variable swap, not a second set of classes (see section 6)
Cache	Cloudflare KV (<code>NEWZ_CACHE</code>)	20-minute TTL on news, 24-hour TTL on fact-check results, stale-on-error fallback
AI reasoning	Workers AI (<code>@cf/meta/llama-3.1-8b-instruct-tf</code>)	Stage 3 of the fact-check pipeline (reasons) over retrieved evidence text, never over open knowledge
News data	NewsData.io API, with an RSS-feed fallback	Category + language filtering; RSS kicks in if the paid API is down or rate-limited

Fact-check sources	Google Fact Check Tools API (stage 1) + Tavily Search (stage 2)	Certified fact-checker lookup first; live web evidence retrieval second
Tests	Vitest (131 tests)	Unit tests on every <code>lib/</code> module, plus a WCAG contrast test that fails the build if a color pair goes unreadable
CI	GitHub Actions	Gates on <code>npm test</code> , <code>npx astro check</code> , <code>npm run build - no true</code> escape hatches

No framework runtime (React/Vue) is shipped to the browser. Interactive bits are plain `<script>` tags scoped per Astro component, compiled to small ES modules.

3. How the Fact-Check Pipeline Actually Works

This is the part of the site that can't afford to be sloppy, so it's worth documenting precisely (source: `src/pages/api/factcheck.ts`).

```
POST /api/factcheck { claim } or { url } | v Stage 1 - Google Fact Check Tools
API Is there already a published, certified fact-check for this claim? -> yes:
return that publisher's rating directly basis = "certified" | no match v Stage
Tavily web search Run a live web search for the claim, fetch the top results
pages' actual text (not just meta descriptions). | v Stage 3 - Workers AI
(ttlama-3.1-8b-instruct-tpa) the model is shown ONLY the claim + the fetched
passages - never asked to answer from its own training knowledge. It returns one
of: verified / misleading / false / insufficient_evidence basis = "ai_assessment"
v If evidence is thin or contradictory anywhere in the chain ->
insufficient_evidence. The system prompt is written specifically so the model
grades the CLAIM, not the passages - a subtle bug class where a debunking passage
not misread as confirming the claim was caught and fixed in testing (documented
in the source comment)
```

Every verdict returned includes the sources it was based on, so a reader can check the reasoning themselves rather than trusting a black box.

Guardrails already tested against real claims:

- "COVID vaccines contain microchips" -> `false`, cites FactCheck.org
- A nonsense claim ("purple goats run the Belgian postal service") -> `insufficient_evidence`, not an invented answer
- Rate-limited at 20 requests/IP/hour to control API cost
- Results cached 24h per claim to avoid re-paying for the same query

4. Design System

Documented in full in [DESIGN.md](#); summarized here:

- **One token source:** colors live in a Tailwind v4 `@theme` block, and an `html.dark { ... }` block re-points the *same* variables. This is why the codebase has almost no `dark:` variants anywhere - `bg-surface-soft` just works in both themes because the variable itself changes.
- **Accessibility contract, enforced by a test:** the brand coral (primary, `#ff6b57`) is only 2.8:1 against white - decorative only, never text. `primary-strong` (`#cc4430`, 4.73:1) is the text-safe version. `tests/contrast.test.ts` asserts every text/surface pair in both themes and will fail CI if a new color combination goes unreadable.
- **The "always-dark-band" exception:** a few UI bands (the breaking-news ticker, the masthead bar, the footer) stay visually dark regardless of site theme. These deliberately use a `dark:` override pointed at a *lighter* fallback token instead of the normal inverting token - otherwise dark mode would put near-black text on a near-black bar. This exact bug shipped once and was caught and fixed (see git history, PR #7).
- **Typography:** Inter for everything; JetBrains Mono for datelines, source labels, and tickers - news metadata reads better tabular.
- **8-point spacing scale,** Tailwind's stock numeric scale (`p-4` = 16px, etc.) - no custom spacing tokens, because in Tailwind v4 a custom `--spacing-*` token silently redefines `max-w-*` too (this collapsed a whole page layout once before the rule was written down).

5. How It Was Built - Step by Step

This mirrors the reference guide's numbered-step format, but describes what actually happened in this repo, not a generic tutorial.

Step 1 - Pick the shape of the product

Two things, not seven: a headline feed and a fact-checker. An earlier iteration of this project had seven stacked homepage widgets (voice TTS, a 1000-line language hub, an admin desk, a chat UI) - all of it was deleted. `DECISIONS.md` in the repo root still describes that abandoned architecture and is explicitly flagged as stale; it is kept only as a historical record, not as documentation.

Step 2 - Scaffold with Astro

```
pnpm create astro@latest && npm install @astrojs/cloudflare @tailwindcss/astro
```

SSR output mode (output: `'server'`) was chosen over static generation because news content changes constantly and category/language are query-driven per request.

Step 3 - Wire up the news feed

`src/lib/news/newsdata.ts` calls `NewsData.io`; `src/lib/news/rss.ts` is the fallback reader used when the API key is missing, rate-limited, or erroring. Both funnel into the same `Article` shape (`src/lib/news/types.ts`) so the rest of the app never has to know which source served a given headline. Category and language are both **allowlists** (`categories.ts`, `languages.ts`) - an unrecognized query param falls back to a default rather than being forwarded upstream. That allowlist is the actual security boundary for those two params.

Step 4 - Build the fact-check pipeline

Built stage by stage (section 3 above), each stage independently testable. `search.ts` originally called Cloudflare's own Web Search binding; it was swapped for Tavily after the Cloudflare binding returned `account_disabled` on every call in production. The `SearchHit` interface didn't change, so stages 1 and 3 didn't need touching - a 5-line diff fixed evidence retrieval site-wide.

Step 5 - Design system and dark mode

Tokens were defined once in `@theme`, then dark mode was implemented as a variable re-point rather than per-component `dark`: classes. This deleted 302 of 307 originally-hardcoded `dark`: variants during the token migration.

Step 6 - The homepage redesign

A visual redesign was prototyped in Claude's Design tool (a separate AI product for mocking up UI), then the resulting HTML/CSS/JS bundle was extracted and re-implemented natively in Astro against the *existing* design tokens and data layer - not copy-pasted, since the mockup used placeholder articles and a client-only reactive framework that doesn't exist in a static-first SSR site. This added: the dark masthead bar, a "Customize topics" popover, per-article Save buttons with a slide-in drawer, a "Most Read" sidebar, a Fact Check promo section, and a floating "Verify a claim" CTA.

Step 7 - Test, then deploy

```
pnpm test # 131 vitest tests npx astro check # 0 type errors, enforced in CI npx
pnpm build # astro build -> dist/ npx wrangler deploy
```

Bindings declared in `wrangler.jsonc`: a KV namespace for caching, a Workers AI binding, and Cloudflare's asset binding for the static output. Secrets (`NEWSDATA_API_KEY`, `GOOGLE_FACTCHECK_API_KEY`, `TAVILY_API_KEY`) are set via `wrangler secret put`, never committed.

Step 8 - CI as a real gate

`.github/workflows/deploy.yml` runs the same three commands as step 7 on every push, with `|| true` bypass - a red test or a type error blocks the pipeline, it doesn't just log a warning.

6. Known Limitations (honest, as of this writing)

- Live at www.newzwale.com (custom domain bound to the Cloudflare Worker via the dashboard, not declared in `wrangler.jsonc`). Both `newzwale.com` and `www.newzwale.com` currently resolve with a 200 and no redirect between them - a canonical redirect (pick one as primary) is worth adding at the Cloudflare DNS/rules level to avoid duplicate-content signals.
- Image/screenshot fact-checking tab exists in the UI, explicitly disabled with "coming soon" - no OCR backend wired up.
- Site *chrome* (nav labels, buttons) stays English regardless of the language selector; only headline content changes language. The control's tooltip says this explicitly rather than over-promising translation.
- No in-site article reading - headlines always link out to the original publisher (a licensing decision, not a technical limitation).
- Fact-check tabs are keyboard-reachable but don't implement the full ARIA roving-tabindex pattern yet.

7. If You Wanted to Monetize This (the angle the reference article takes)

The `compilefuture.com` piece frames a site like this as a **micro-tool + SEO play**: build something genuinely useful, let organic search traffic find it, and monetize with display ads (AdSense) once there's enough volume to matter. The same shape applies here, with the caveat that these are general patterns, not numbers this specific site has earned:

- **The tool itself is the acquisition channel.** A working fact-checker for a specific claim ("is X true") is exactly the kind of long-tail, high-intent query search engines reward - someone searching "is [claim] true" has strong intent to click.
- **AstroJS matters for this**, same reasoning as the reference article: SSR-by-default with near-zero shipped JS means fast Lighthouse scores out of the box, which is a direct SEO input.
- **The realistic timeline** for this kind of niche content/tool site to build meaningful organic traffic is measured in months, not days - the reference guide cites 4-6 months as a typical inflection point for a well-executed micro-tool. That is a general pattern for this category of site, not a projection for NewzWale specifically.
- **Ad placement** would sit naturally around the headline feed and the verdict result - high-attention areas - without interrupting the actual fact-check flow.

This section is deliberately hedged: it describes *how sites like this typically monetize*, not a promise about what this specific deployment will earn.

8. Quick Reference

```
pm install # install dependencies npm run dev # local dev server, localhost:4321
pm run build # production build to dist/ npm run preview # build + run under
wrangler dev (closer to prod) npm test # run the vitest suite npx astro check #
type-check the whole project npm run deploy # build + wrangler deploy
production.
```

Sources

- [DESIGN.md](#) - full design system reference
- [PROGRESS.md](#) - living build log, session handoff notes, current gotchas
- [DECISIONS.md](#) - **stale**, describes an abandoned earlier architecture, kept for history only